Data, RAG & Power BI

Grounding AI in trusted data, and turning clean data into insight

Data quality for AI | Retrieval-augmented generation | Azure AI Search patterns | Power BI basics

Garbage in, garbage out

Answers with citations

Dashboards people trust

CodeSizzler · Microsoft Preferred Partner, Data & AI
www.codesizzler.in

Five Days from Cloud Basics to Production-Ready AI

Each day adds one layer. Every day mixes short concepts, a live demo and a hands-on lab.

DAY 1

Azure & Agent Foundations

Agentic workflows with Microsoft 365 Copilot

- Azure cloud foundations
- How AI agents work
- Agent Builder, Notebooks, Cowork, Skills
- Chaining agents with a human in the loop

DAY 2

Copilot Studio

Low-code and pro-code conversational AI

- Agents, workflows and apps
- Knowledge, MCP tools, skills
- Topics and the standard harness
- Evaluate, publish, govern

DAY 3

Microsoft Foundry

Azure AI foundation and core concepts

- Projects, resources, endpoints
- Model catalog and comparison
- Prompting and evaluation
- Secure, private, observable agents

DAY 4 · TODAY

Data, RAG & Power BI

Grounding AI in trusted data

- Data quality for AI
- Retrieval-augmented generation
- Azure AI Search patterns
- Power BI basics and dashboards

DAY 5

Microsoft Fabric

Enterprise analytics with Copilot

- OneLake, lakehouse, shortcuts
- Copilot in Fabric and Power BI
- Fabric IQ ontology and data agents
- Governance and the road ahead

Foundations → Microsoft 365 Copilot → Copilot Studio → Foundry → RAG & Power BI → Fabric

Today: Clean Data, Grounded Answers, Clear Dashboards

Three parts. Two labs with a browser track and a local Python track.

01 Data for AI | 02 RAG concepts | 03 Azure AI Search | 04 Power BI basics

Lab: Data Fundamentals

- Generate a messy dataset
- Measure, profile, clean
- Compare before vs after
- Classify sensitive data

Concepts + demo

- Why models need grounding
- Ingestion and query pipelines
- Chunking, embeddings, hybrid search

Architecture

- Index, indexer, skillset
- Citations and security trimming
- Evaluating RAG

Lab: Power BI

- Clean data in Power Query
- Semantic model and DAX
- Two dashboard levels, then share

You need

- Python 3 with pandas and faker
- Fabric workspace ws-p1-<your user name>
- Power BI trial (start once)
- raw_customer_data_large.csv

The same customer dataset runs through both labs: cleaned in Python, then shaped and visualised in Power BI.

01

Data Fundamentals for AI

Why data quality decides AI quality, measured with real numbers

Quality dimensions | The messy dataset | Profile & clean | Before vs after | Governance

Garbage In, Garbage Out, Now at AI Speed

AI does not fix bad data. It repeats it confidently, at scale.

01 Accuracy

Values are correct. A negative annual spend is not.

02 Completeness

Required values are present. Missing dates break trend analysis.

03 Consistency

One meaning, one spelling. “USA”, “usa” and “United States” are one country.

04 Validity

Values follow the rules and formats, e.g. one date format.

05 Uniqueness

Each real-world thing appears once. Duplicate customers inflate counts.

06 Timeliness

Data is current enough for the decision being made.

An agent answering from dirty data gives wrong answers with perfect grammar.

A Dataset Built to Be Messy

A Python script generates 2,063 customer rows with real-world problems planted on purpose.

01 Five date formats

01/31/2023, 31-01-2023, 23-01-31… all in one SignupDate column.

02 Country chaos

32 different labels for only 10 real countries: USA, usa, U.S.A., United States…

03 Hidden duplicates

Repeated customers each get a new CustomerID, so they look unique.

04 Invalid values

Negative and missing AnnualSpend values.

05 Messy text

Random upper and lower case, inconsistent phone formats.

06 Two tracks

Load into a Fabric lakehouse, or a local SQLite “lakehouse” with Python.

Columns: CustomerID · Name · Email · SignupDate · Country · AnnualSpend · Phone

Three Bugs That Look Like Success

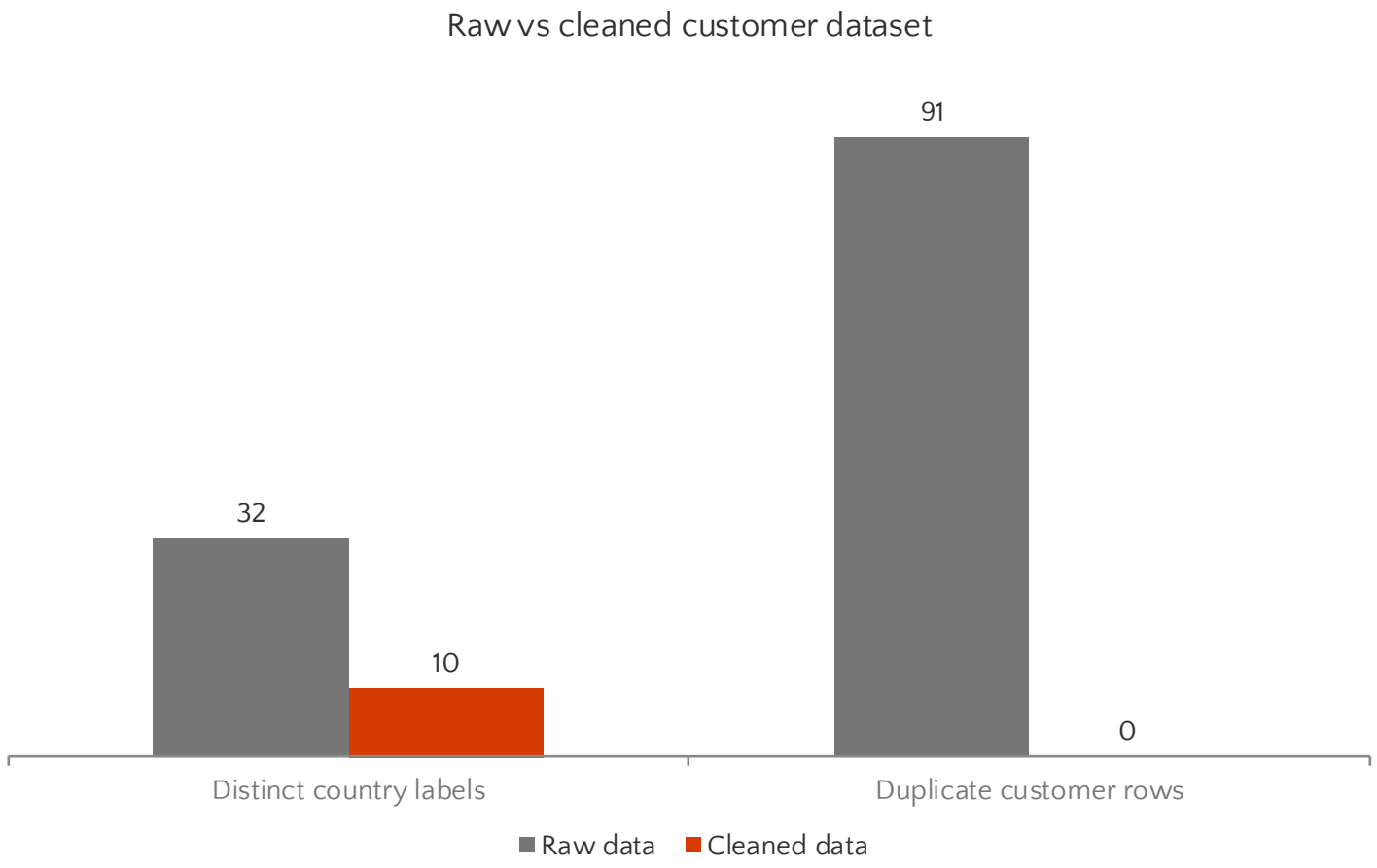
Each one was found by running the code, not by reading it.

Trap	What the naive code did	What actually works
Mixed dates	✕ pd.to_datetime(dayfirst=True) silently blanked 1,653 of 2,063 dates (80%)	✓ Add format="mixed" so each value is parsed on its own: 0 blank dates
Hidden duplicates	✕ drop_duplicates() on the full row removed 0 rows	✓ De-duplicate on the business key: Email, case-insensitive
Country spellings	✕ A hand-written map missed variants like a trailing space	✓ Lower-case and trim first, then map: complete by construction
Ambiguous dates	✕ 03/04/2023 could be 3 April or 4 March	✓ Capture one consistent format at the source

No error message appeared for any of these. Always profile the result, not just the code.

Cleaning Changed the Answers

Tested reference numbers from one run. Your figures should land very close.



OTHER CHANGES

- **Rows:** 2,063 → 1,972
- **Average spend:** ≈ 2,402.98 → ≈ 2,411.98
- **Blank dates:** 0 after the fix
- **Negative spend:** flagged, not silently altered

An AI or report on raw data would have counted 32 countries.

Classify Before You Share: Personal and Sensitive Data

Part 6 of the lab: decide, column by column, how each field must be handled.

Column	Personal data?	Handling
Name	Yes	Mask or remove before sharing outside the owning team
Email	Yes	Use as a key internally; never expose in reports or agent answers
Phone	Yes	Restrict access; standardise format
Country	No (on its own)	Safe for aggregated reporting
AnnualSpend	Business-sensitive	Aggregate in dashboards; limit row-level access
CustomerID	Indirectly	Pseudonymous key; still protect when combined with other fields

Label it with Microsoft Purview; enforce it with permissions; keep it out of prompts unless needed.

02

Retrieval-Augmented Generation (RAG)

How agents answer from your documents, with citations

- Why grounding
- Two pipelines
- Chunking
- Embeddings
- Hybrid search

Why Models Need Grounding

A model on its own knows a lot about the world, and nothing about SATORP's procedures.

01 Knowledge cut-off

Models are trained up to a date. Your SOP updated last month is unknown to them.

02 Private data

Your procedures, specs and reports were never in the training data, and shouldn't be.

03 Hallucination

Without a source, a model fills gaps with plausible-sounding guesses.

04 Trust

Engineers need to see where an answer came from before they act on it.

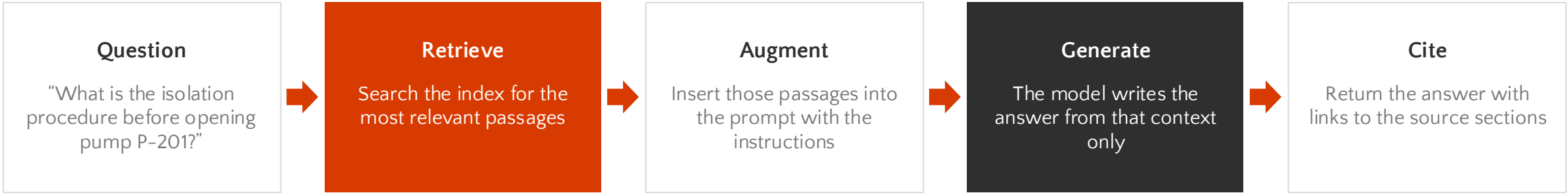
TWO WAYS TO ADD KNOWLEDGE

- **RAG:** look up relevant passages at question time. Fresh, cited, permission-aware.
- **Fine-tuning:** retrain the model on examples. Changes style and behaviour, not facts.

For facts in documents, start with RAG.

Retrieve, Augment, Generate

Three steps at question time. The model only writes after the search has found the evidence.



The prompt the model sees

Instructions + the user's question + the retrieved passages, each labelled with its source.

The key instruction

"Answer only from the sources. If the answer isn't there, say you don't know."

Most failures are retrieval failures

If search returns the wrong passages, even the best model gives a wrong answer.

Two Pipelines: Prepare Once, Answer Many Times

Ingestion runs in the background. Query runs every time someone asks.

Ingestion pipeline (offline)

Turn documents into a searchable index.

- Collect: SharePoint, Blob Storage, OpenText
- Crack: extract text from PDF, Word, scans (OCR)
- Chunk: split into passages
- Embed: turn each chunk into a vector
- Index: store text, vectors and metadata
- Refresh on a schedule or on change

Query pipeline (online)

Answer a question in seconds.

- Receive the question and the user's identity
- Search: keyword + vector (hybrid)
- Re-rank with the semantic ranker
- Filter to what this user may see
- Build the prompt, generate, cite

Better ingestion beats a bigger model. Most RAG quality is decided before the question is asked.

Chunking: How You Cut Documents Matters

Chunks are the units the search finds. Too big and they're vague; too small and they lose context.

01 Fixed size

Split every N tokens, e.g. a few hundred. Simple and predictable; may cut mid-sentence.

02 Overlap

Repeat a little text between neighbouring chunks so ideas aren't cut in half.

03 By structure

Split on headings and sections. Best for SOPs and manuals with clear structure.

04 Keep metadata

Store the title, section, revision and URL with each chunk for filtering and citations.

05 Tables and drawings

Extract tables carefully; P&IDs and scanned forms may need Document Intelligence.

06 Test, don't guess

Try two chunk sizes on your real questions and compare retrieval quality.

For SATORP SOPs: chunk by section, keep the revision number, and cite the section.

Embeddings: Searching by Meaning

An embedding turns text into a list of numbers that captures its meaning.

Texts with similar meaning get similar numbers, so search can find them even when the words differ.

- **Example:** “compressor trip” and “compressor shut down unexpectedly” land close together.
- **Vector search:** find chunks whose embeddings are nearest to the question’s embedding.
- **Embedding model:** a separate, small model deployed in Foundry, e.g. a text-embedding model.
- **Same model both sides:** embed documents and questions with the same model.
- **Weakness:** exact codes like “P-201” or “AST-2001” are better matched by keyword search.

ANALOGY

Like a map of meaning.

- Every chunk is a pin on the map
- The question is a new pin
- Search returns the nearest pins

Meaning for concepts; keywords for codes.

Keyword, Vector, Hybrid and Semantic Ranking

Azure AI Search can combine them. Hybrid plus semantic ranking is the usual best starting point.

Approach	How it works	Strong at	Weak at
Keyword (full text)	Matches the words in the question	Tags, codes, exact phrases	Synonyms, paraphrases
Vector	Finds chunks with the nearest embeddings	Meaning, paraphrases	Exact codes and numbers
Hybrid	Runs both and merges the rankings	Both at once	Slightly more cost
+ Semantic ranker	Re-orders the top results with a language model	Putting the best passage first	Adds a little latency

Default for SATORP documents: hybrid search + semantic ranker + metadata filters.

Inside Azure AI Search

Five objects you configure. Integrated vectorisation can chunk and embed for you during indexing.

Data source	Where documents live: Blob Storage, ADLS, SharePoint, SQL, OneLake
Indexer	Pulls documents on a schedule and tracks changes
Skillset	Enrichment steps: OCR, chunking, calling an embedding model
Index	Searchable fields: text, vectors, metadata, security fields
Query	Keyword, vector or hybrid, with filters and semantic ranking

Where you meet it this week	
Day 1	The engine behind grounding
Day 2	Copilot Studio knowledge (managed for you)
Day 3	A tool for Foundry agents
Day 4	RAG architecture, today
SAP guide	Index of OpenText documents
Security	Private endpoint, RBAC roles

Security in RAG: Two Risks to Design For

A search index is a copy of your documents. Treat it with the same care.

01 Leaking via search

If everyone can search everything, the agent can reveal documents a user shouldn't see.

02 Security trimming

Store who may see each document (groups) in the index; filter every query by the user's identity.

03 Prompt injection in documents

A document can contain hidden text like "ignore your instructions". The model may obey it.

04 Defences

Prompt Shields for indirect attacks, approved sources only, and tests with poisoned documents.

CHECKLIST

- Index only approved, owned sources
- Carry permissions into the index
- Private endpoint and RBAC on the search service
- Include injection cases in the test set

The agent never sees more than the user asking.

When RAG Gets It Wrong: Symptoms and Fixes

Check retrieval first. Most problems are found there.

Symptom	Likely cause	Fix
Answer is vague or generic	Chunks too large; wrong passages retrieved	Smaller, section-based chunks; hybrid + semantic ranker
Misses equipment tags or codes	Vector-only search	Add keyword search (hybrid)
Cites an outdated procedure	Old versions still indexed	Remove superseded documents; filter by revision
Confident answer, no source	Weak instructions	"Answer only from sources, else say you don't know"
Wrong user sees a document	No security trimming	Permission fields and query filters

Evaluating a RAG System

Measure retrieval and generation separately, so you know which half to fix.

Retrieval quality

Did search find the right passages?

- Is the correct section in the top results?
- Are irrelevant chunks crowding it out?
- Test with real user questions and known answers

Answer quality

Did the model use them well?

- Groundedness: supported by the sources
- Relevance: answers the question
- Completeness: nothing important missing

Safety and trust

Can people rely on it?

- Citations are correct and clickable
- Refuses when the answer isn't there
- No leaks across permissions
- Resists injected instructions

Use the Foundry evaluators from Day 3: groundedness and relevance are the core RAG metrics.

You Don’t Always Build RAG Yourself

Each Microsoft platform offers RAG at a different level of control.

Platform	How RAG works there	You control
Microsoft 365 Copilot	Grounds on Microsoft Graph: your mail, files, Teams and SharePoint, with your permissions	Which sources an agent may use
Copilot Studio	Knowledge sources: SharePoint, files, websites, Dataverse; retrieval managed for you	Sources, instructions, moderation
Microsoft Foundry	Azure AI Search as an agent tool, or your own RAG code	Everything: chunking, search, prompts
Microsoft Fabric	Data agents answer from tables in OneLake (structured, not documents)	Sources, instructions, example queries

Same rule as Day 1: start with the most managed option, go deeper only when you need control.

03

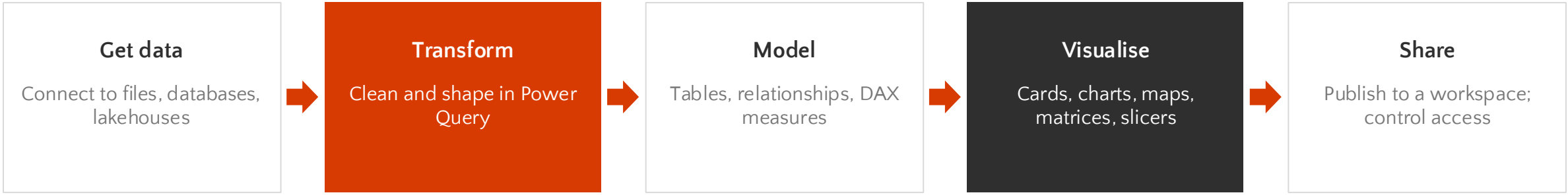
Power BI Basics

From messy CSV to a dashboard people trust

Power Query | Semantic model | DAX measures | Dashboards | Sharing

Five Steps from Data to Decision

Every Power BI report follows the same path.



Power BI Desktop

Windows app for authoring. The most features.

Browser track

Power Query Online in a Dataflow Gen2, and reports in the Power BI service in Fabric. Same learning.

Copilot in Power BI

Builds report pages and summaries from a semantic model (Day 5).

Cleaning the Data in Power Query

The same data rescue as this morning, without writing code. Dataflow Gen2: Customer_Data_Cleanup.

Ingest	Import from Text/CSV; upload raw_customer_data_large.csv
Standardise countries	Capitalize Each Word, then Replace Values for the USA variants
Fix spend	Filter out nulls; Number Filters → Greater Than 0
Remove duplicates	Home → Remove rows → Remove duplicates; compare row counts
Save to lakehouse	Destination: Lakehouse table customers_clean; Save and run

Watch out	
USA	Five spellings: USA, usa, U.S.A., …
“Of”	Capitalize changes “of” to “Of”
Check	Country filter list afterwards
Counts	Note rows before and after
Workspace	Use ws-p1-<user>, don’t create one

The Semantic Model and DAX Measures

A semantic model is the business layer: tables, relationships and measures everyone reuses.

Why measures matter

- **Defined once:** every report and Copilot uses the same definition.
- **Calculated on the fly:** they respond to filters and slicers.
- **Same DAX** in Desktop and in the browser.
- **Star schema:** facts in the middle, dimensions around them, as the model grows.

DAX measures from the lab

```
Total Customers =
    DISTINCTCOUNT(customers_clean[CustomerID])

Total Spend =
    SUM(customers_clean[AnnualSpend])

Average Spend =
    AVERAGE(customers_clean[AnnualSpend])

High Value Customers =
    CALCULATE(
        DISTINCTCOUNT(customers_clean[CustomerID]),
        customers_clean[AnnualSpend] > 5000
    )
```

Two Dashboard Levels, Then Share

Start with a quick win for executives, then add depth for regional managers.

Level 1 · Executive Overview

The quick win

- Three cards: Total Customers, Total Spend, Average Spend
- Map: Country by Total Spend
- Bar chart: spend by country, sorted

Level 2 · Deep dive

For regional managers

- Matrix: Country × Spend and Customers
- Conditional formatting on spend
- High Value Customers measure
- Country slicer to filter the page

Part 4 · Sharing

Get it to the right people

- Save in the workspace
- Share with specific people or groups
- Respect row-level access for sensitive data

Clean data + one semantic model + clear visuals = numbers people stop arguing about.

What We Covered Today

Clean data and good retrieval decide AI quality more than the model does.

32 → 10
country labels after cleaning

91
hidden duplicate customers removed

Hybrid
keyword + vector + semantic ranking

2 levels
of Power BI dashboard on clean data

Remember

- Profile the result, not just the code
- Most RAG failures are retrieval failures
- Chunk by section, keep metadata, cite it
- Carry permissions into the index

Coming up on Day 5: Microsoft Fabric

- OneLake, lakehouses and shortcuts
- Copilot in notebooks and Power BI
- Fabric IQ: a refinery ontology
- Fabric data agents in plain language

DAY 4 COMPLETE

Thank You. Questions?

Microsoft Data & AI Readiness for SATORP · Day 4: Data, RAG & Power BI

NEXT

Day 5: Microsoft Fabric & enterprise analytics. You need a workspace on a paid Fabric capacity (F2 or higher) with Contributor access; trial capacities don't support Copilot.

info@codesizzler.in
www.codesizzler.in · +91 90801 16454